

Flatten a Linked List

Complete Personal DSA Notes — intuition, required code comments, recursion dry run, and clear complexity

PDF instructions applied: explain intuition before code; comment only important pointer/boundary lines; explain time and space complexity clearly; and for recursion include intuition, base case, recursive flow, backtracking, and dry run.

1. Problem Statement

The input is a special linked-list configuration. Every node has a **next** pointer and a **child** pointer. The horizontal next-chain contains sorted vertical child-linked lists. Flatten everything into one sorted vertical list using **child**, with the final nodes' **next** pointers set to null.

```
Horizontal: 1 → 2 → 3 → 4 → 5
             ↓   ↓
             7   10
             ↓
             11
             ↓
             12
```

Flattened: 1 ↓ 2 ↓ 3 ↓ 4 ↓ 5 ↓ ... ↓ 12 ↓ NULL

Each vertical child list is already sorted. That observation is the key to the optimized solution.

2. Intuition

Brute force: collect every value, sort all values, then build a new vertical list. Easy, but it ignores the fact that the child lists are already sorted and uses extra space.

Optimized: flattening two sorted child lists is just the merge step from merge sort. So merge two lists, then recursively merge that result with the previous horizontal list. Reuse the same nodes instead of creating new ones.

```
L1 + L2 → merged
merged + L3 → merged
merged + L4 → merged
...
```

Recursion goes right first.
Merging happens while returning (backtracking).

3. Brute Force Approach

```
for (temp = head; temp != null; temp = temp.next) {
    for (temp2 = temp; temp2 != null; temp2 = temp2.child) {
        arr.add(temp2.data);
    }
}
sort(arr);
head = convertToVerticalList(arr);
```

Under the supplied video's simplified assumption, there are n horizontal nodes and each vertical list has M nodes, so there are $n \times M$ total values.

4. Brute Force Complexity

Collecting values costs $O(nM)$. Sorting $X=nM$ values costs $O(X \log X) = O(nM \log(nM))$. Rebuilding the vertical list costs $O(nM)$. Therefore the dominant time is **$O(nM \log(nM))$** . Extra space is **$O(nM)$** for storing all values (and the new output nodes if counted as newly allocated storage).

```
Brute Force:
Time →  $O(nM \log(nM))$ 
Space →  $O(nM)$ 
```

5. Optimization Insight

Two things can be removed: **sorting**, because every vertical list is already sorted; and **new-node creation**, because the existing nodes can be linked in place.

6. Merge Two Sorted Child Lists

Keep two pointers. Compare their values, attach the smaller existing node through **child**, move that pointer, and continue. When one list ends, attach the remaining part of the other list.

```
L1: 4 ↓ 9 ↓ NULL
L2: 5 ↓ 6 ↓ 8 ↓ NULL
```

```
4 < 5 → take 4
9 > 5 → take 5
9 > 6 → take 6
9 > 8 → take 8
L2 ends → attach 9
```

```
4 ↓ 5 ↓ 6 ↓ 8 ↓ 9 ↓ NULL
```

This resembles merge-two-sorted-lists, but the result is connected with **child**, not next.

7. Dummy Node Intuition

A dummy node avoids a special case for the first result node. At the end, **dummy.child** is the real merged head.

```
ListNode dummy = new ListNode(-1);
ListNode result = dummy;

while (list1 != null && list2 != null) {
    if (list1.data < list2.data) {
        result.child = list1;
        result = list1;
        list1 = list1.child;
    } else {
        result.child = list2;
        result = list2;
        list2 = list2.child;
    }
}
result.child = (list1 != null) ? list1 : list2;
return dummy.child;
```

8. Important Pointer Rule

The flattened list uses child links. Therefore the old next links must not remain in the flattened structure. When a node is added to the result, set its next to null.

```
result.child = list1; // reuse existing node
result = list1;
list1 = list1.child; // move through child
result.next = null; // final structure is vertical
```

9. Two-List Merge Code — Required Comments

```
private ListNode merge(ListNode list1, ListNode list2) {
    ListNode dummy = new ListNode(-1);
    ListNode result = dummy;

    while (list1 != null && list2 != null) {
        if (list1.data < list2.data) {
            result.child = list1;
            // Reuse existing list1 node.
            result = list1;
            list1 = list1.child;
        } else {
            result.child = list2;
            // Reuse existing list2 node.
            result = list2;
            list2 = list2.child;
        }

        result.next = null;
        // Remove horizontal link from flattened node.
    }

    result.child = (list1 != null) ? list1 : list2;
    return dummy.child;
}
```

10. Recursion Intuition

There are many horizontal lists, so solve the smaller problem first: recursively flatten everything to the right. When that recursive call returns, it gives the current list a completely flattened sorted list to merge with.

```
1 → 2 → 3 → 4 → 5
      ↓
    base case
```

```
return:
merge(4,5)
merge(3, merged(4,5))
merge(2, merged(...))
merge(1, merged(...))
```

The important idea is: **go deep first, merge while backtracking.**

11. Base Case

```
if (head == null || head.next == null) {
    return head;
    // Only one horizontal list remains.
}
```

12. Complete Recursive Java Code

```
class Solution {

    private ListNode merge(ListNode list1, ListNode list2) {
        ListNode dummy = new ListNode(-1);
        ListNode result = dummy;

        while (list1 != null && list2 != null) {
            if (list1.data < list2.data) {
                result.child = list1;
                // Reuse existing node.
                result = list1;
                list1 = list1.child;
            } else {
                result.child = list2;
                // Reuse existing node.
                result = list2;
                list2 = list2.child;
            }

            result.next = null;
            // Final list is vertical.
        }

        result.child = (list1 != null) ? list1 : list2;
        return dummy.child;
    }

    public ListNode flattenLinkedList(ListNode head) {

        if (head == null || head.next == null) {
            return head;
            // Base case.
        }

        ListNode mergedHead = flattenLinkedList(head.next);
        // Go to the last list first.

        return merge(head, mergedHead);
        // Merge during backtracking.
    }
}
```

13. Recursive Dry Run

For a small horizontal chain $1 \rightarrow 2 \rightarrow 3$:

```
flatten(1)
→ flatten(2)
→ flatten(3)
```

$3.next == null \rightarrow \text{return } 3$

Back at 2:
 $mergedHead = 3$
 $merge(2, 3) \rightarrow \text{return merged head}$

Back at 1:
 $mergedHead = merged(2,3)$
 $merge(1, mergedHead) \rightarrow \text{final head}$

This is backtracking because after the recursive call returns, the merge operation is performed before returning from the current call.

14. Recursive Pointer Picture

```
DESCENT:
1 → 2 → 3 → END
^         ^
first    base case

RETURN:
3
↓
merge(2,3)
↓
merge(1, merged(2,3))
↓
final flattened head
```

15. Why the Recursive Solution Works

At each return, everything to the right is already one sorted vertical list. The only remaining job is to merge the current vertical list with that returned list. Repeating this while backtracking eventually produces one sorted vertical list.

16. Time Complexity — Following the Video's Analysis

The supplied material assumes n horizontal lists and M nodes in each vertical list. Recursion goes through about n horizontal levels, and a merge is performed at each returning level except the last. A merge of two lists costs $O(N_1 + N_2)$; under the simplified equal-length assumption, this is $O(2M)$. Therefore the video-style analysis gives $O(n \times 2M) = O(2nM)$, which is asymptotically $O(nM)$.

Recursion levels	→ about n
Merge per level	→ $O(2M)$ under assumption

Total	→ $O(2nM)$
Asymptotically	→ $O(nM)$

The exact merge cost depends on the sizes of the lists being merged; M is the simplifying assumption used in the supplied material.

17. Space Complexity — Following the Video's Analysis

The merge reuses existing nodes and uses only constant pointer variables. The extra space comes from the recursive call stack. With n horizontal levels, the maximum recursion depth is n , so auxiliary space is $O(n)$.

Existing nodes reused in place	→ no new list storage
Recursion stack	→ $O(n)$

Auxiliary Space → $O(n)$

18. Brute Force vs Optimized

Brute Force
collect + sort + rebuild
Time → $O(nM \log(nM))$
Space → $O(nM)$

Optimized Recursive
merge sorted lists + reuse nodes
Time → $O(2nM)$ under video assumption
→ $O(nM)$ asymptotically
Space → $O(n)$ recursion stack

19. Edge Cases

- `head == null` → return null.
- `head.next == null` → already one vertical list.
- One vertical list is exhausted during merge → attach the remaining list.
- Duplicate values → merge still remains sorted.
- Final flattened structure → child carries the list and next should be null.

20. Most Important Lines to Remember

```

if (head == null || head.next == null)
    return head;          // Base case

ListNode mergedHead =
    flattenLinkedList(head.next); // Go deep

return merge(head, mergedHead); // Backtrack + merge

result.child = list1;      // Use child
list1 = list1.child;      // Move vertically
result.next = null;       // No horizontal link

```

21. Quick Revision

Brute: collect → sort → rebuild.

Optimized: use sorted property → merge two child lists → recurse over horizontal lists → reuse nodes.

Recursion: go deep → base case → return → merge during backtracking.

Complexity: video-style recursive analysis $O(2nM)$, asymptotically $O(nM)$; auxiliary space $O(n)$.